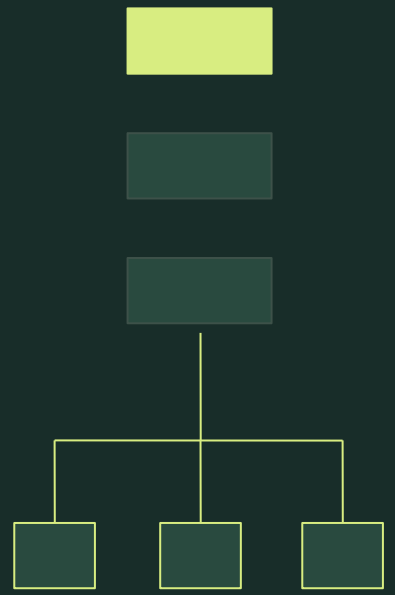
從一人 AI 工作流
到團隊工程系統



HAO

功能完成了。
驗證也過了。

這就夠了嗎？

PULL REQUEST

- 功能行為
- 既有驗證

維護成本？

產出加速時，維護成本也可能一起增加

當只在意「把這個功能做完」，三種代價容易被忽略。

01 單檔膨脹

更多責任
擠進同一個檔案

02 複製貼上

同一個概念
長出多套版本



03 回歸難查

改 A 壞 B
影響範圍更難懂



省下的實作時間，可能不會比後續修正、除錯、審查的時間還短。

案例 01 / Cursor-based pagination 實作

這次改得少，不代表未來維護得少

既有 API 各自實作一套；AI 參考現況再複製一份。

既有 API A / B / C

```
interface UserPage {
  items: User[];
  nextCursor?: string;
}
interface UserCursorCodec {
  encode(value): string;
  decode(token): Cursor;
}
```

新增 API D

```
interface FilePage {
  items: File[];
  nextCursor?: string;
}
interface FileCursorCodec {
  encode(value): string;
  decode(token): Cursor;
}
```

行為可以正確

仍留下重複的責任

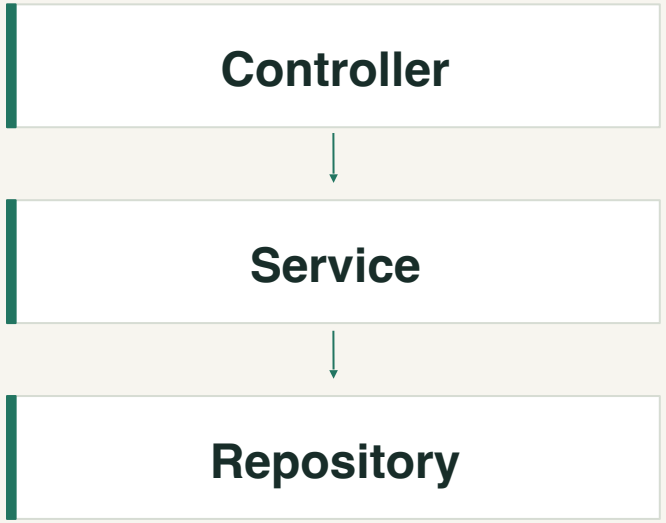
完成當下任務不等於做出長期合理的工程決策。

案例 02 / 模組依賴

沒有設置依賴關係的限制，可能發生錯誤的依賴關係

Agent 不知道依賴關係規則，所以有可能出現不預期的反向依賴

本來的期待



不應出現的依賴



編譯通過，不代表模組的依賴關係是正確的。

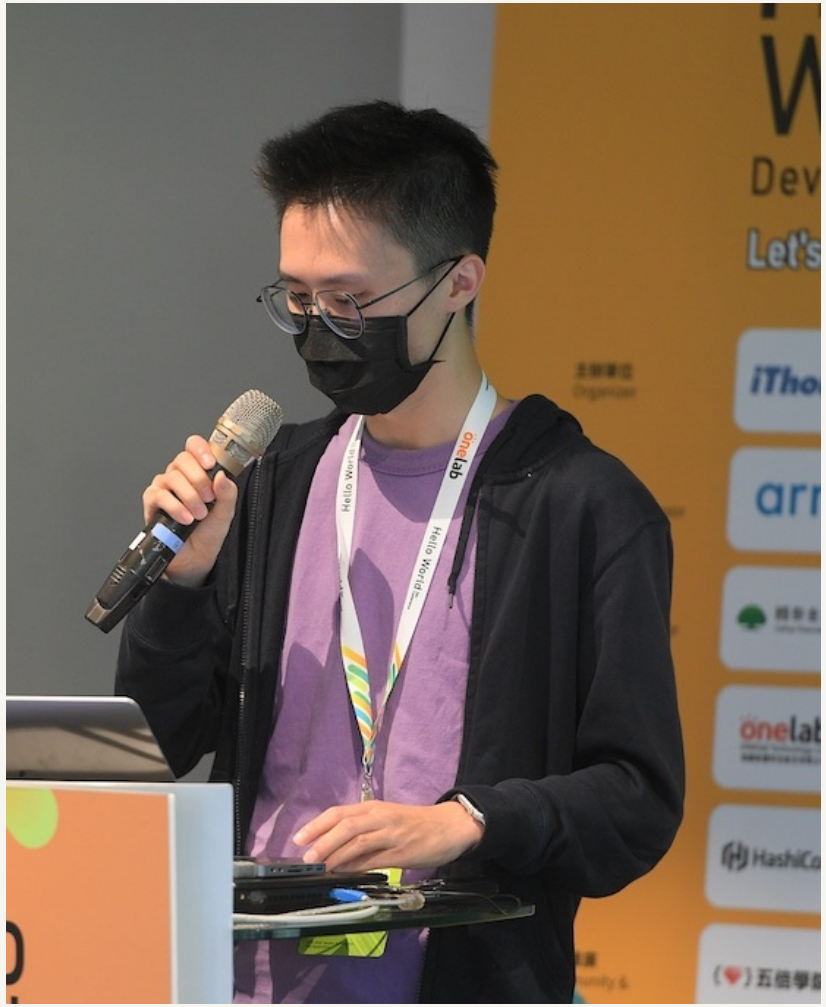
這真的是我們
期望的開發模式嗎？

更多程式碼 ≠ 更快交付

把工程紀律，
做進 Agent 的工作環境。

讓品質可被驗證，
讓有效做法有機會被團隊重用。

About me



HAO

TSMC IT Senior Engineer

作品與分享

《NestJS 基礎必學實務指南》

「用 NestJS 闖蕩微服務！」

「NestJS 帶你飛！」

「今晚我想來點 Express 佐 MVC 分層架構」

「區塊練起來 - 智能合約與 DApp 開發」

今天的三大主軸

對齊目標、建立回饋，再共享經驗。

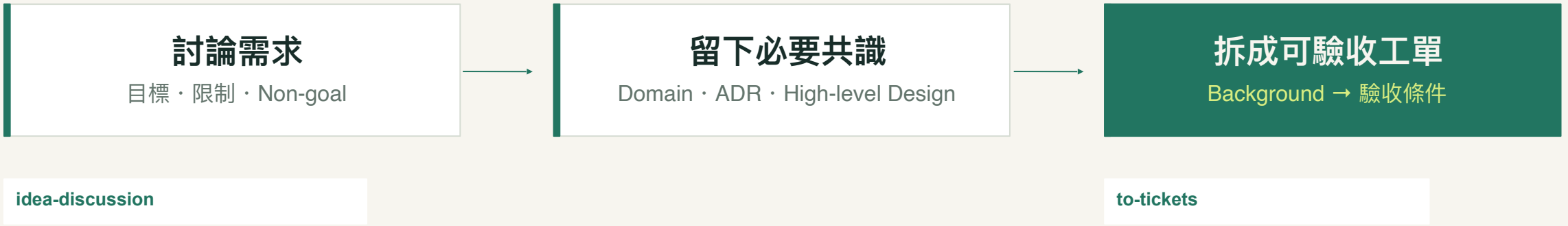
01 **如何先對齊目標？**
需求、語言與可驗收工單

02 **如何及早發現偏離？**
規則、工具、測試與 Review

03 **如何讓下一個人不用重新踩坑？**
共享、分發與治理

從先寫程式改成先取得共識

把需求討論、決策與拆單，變成可以重複執行的流程。



依複雜度、模型強度決定流程細節。

先取得共識，再讓 Agent 大量產出。

將重要的共同語言留下

對話結束，不應該讓重要名詞與決策一起消失。

Domain Language

我們說的是同一個東西

HAO point 是會員點數
Super HAO 是管理員

ADR

我們為何做這個選擇

採用方案、替代方案
與接受的 trade-off

High-Level Design

系統如何分工

服務責任、資料所有權
與模組關係

留下的重要 Context 可以避免新的一輪對話完全失憶

工單要定義完成條件

示例：為文件清單提供 cursor-based pagination。

Background

清單持續增加，需要分頁續讀。

Goal

以 cursor 取得下一頁。

Non Goal

不改授權規則或資料存取層。

Acceptance Criteria

Given	固定資料集 [A, B, C, D]，每頁 2 筆
When	使用第一頁的 nextCursor 續讀
Then	得到 [C, D]，沒有重複；尾頁無 nextCursor

工單提供的資訊與驗收條件將是 Agent 實作時的重要依據

兩個開源專案例子

工具可以換，先釐清再實作的思路不變

它們的共同點是把流程變成可重複的方法。

Superpowers

先討論，再規劃

brainstorming
writing-plans

Mattpocock Skills

先對齊，再拆單

grill-with-docs
to-spec / to-tickets

重點是「先釐清再實作」的流程與思路，不一定要採用哪一套框架。

有了共識，還需要明確的開發規則

需求告訴 AI 做什麼；Policy 定義這個專案必須遵守什麼。

01 **Pagination 必須使用共用層，不得複製、重複實作。**

02 **Service 不得依賴 Controller。**

03 **依責任拆分檔案，遵循既定目錄結構。**

> **只把「重點」落地成 Policy，讓 AI 知道規則是什麼就好。**

回到案例 01

這次先抽共用，下一次把規則留下來

先把合理的共用能力做出來，再要求後續實作遵循它。

Before

- **API A** — 各自維護 interface + codec
- **API B** — 各自維護 interface + codec
- **API C** — 各自維護 interface + codec

After

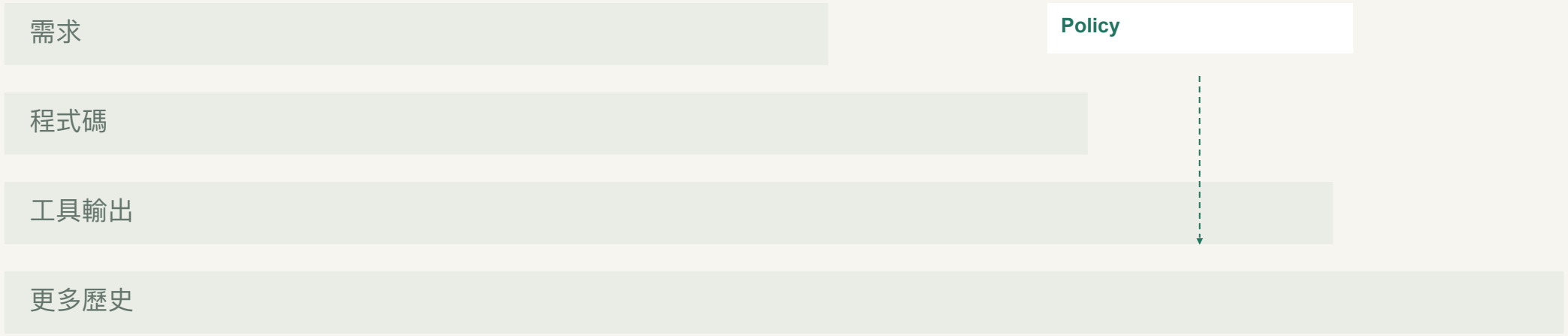


各 API 保留自己的查詢與領域責任

Fix the Code → 留下使用共用層的 Policy → 改善下一次。

Policy 寫下來，不代表每次都會被遵守

長任務與 Context 管理，可能讓重要規則被忽略。



看得到 ≠ 保證會執行

> **能用工具驗證的規則，就不要只靠 AI 會不會記住。**

能確定性檢查的規則，就交給工具

以模組依賴為例：從期待轉成可以執行的約束。

定義依賴規則
Service 不能依賴 Controller

→

分析 dependency graph
Linter 或自製 Checker

→

回報違規與位置
違規時擋下來

```
forbidden:  service  ->  controller
rule:       no-service-to-controller
result:     FAIL
```

工具可以保護已定義、已覆蓋、確定性高的規則。

讓 Commit 的失敗，變成下一步可用的回饋

本地提早擋下；重要檢查再由 CI 與合併規則把關。

工具輸出示意

```
$ git commit
[boundary-check] FAIL
from: services/file-service.ts
to:   controllers/file-controller.ts
rule: service must not depend on controller
exit: 1  (commit aborted)
```



本地 hook 可被略過；不要把它當作不可繞過的保證。

不同問題，需要不同的驗證方式

但不是所有工程判斷，都能寫成 Linter

依賴邊界

這條 import 合法嗎？

規則明確，可直接檢查依賴圖。

是否值得共用

它們真的是同一個責任嗎？

需要理解語意、責任與變動方向。

需要判斷哪些適合用工具處理、哪些不適合

Reviewer Agent

Reviewer 不只說有問題，還要提出證據

code-review Skill 定義檢視方法；Agent 負責取得 Context、檢查與回報。

PR diff

Policy

Codebase

→

code-reviewer

讀取 → 檢查 → 回報

Finding 示意

規則	PG-01：使用既有共用層
證據	新增 codec 與 shared/pagination 職責重複
影響	cursor 語意可能分歧，增加多處維護

用定義好的 Policy 由 Agent 指出改動違反了什麼 Policy

AI Review 是一道防線，但不表示 PR 通過

AI 可能漏判、誤判；規範與證據讓人能檢查它的判斷。

工具	**已定義的明確規則** 型別、Lint、依賴與測試
Reviewer Agent	**需要 Context 的檢視** Policy、重複實作、效能風險、驗收條件
人類	**最終判斷與責任** 商業邏輯、架構取捨、風險接受

> **AI 提供 findings，最終 PR 核准建議還是交由人類判斷。**

要減少改 A 壞 B，先把驗收條件變成測試

先列情境、確認預期，再一次實作一個可驗證的行為。

工單驗收條件 → 測試情境 → **人與 Agent 確認預期**

RED

確認測試因
缺少行為而失敗

GREEN

實作到
該行為通過

REFACTOR

在測試保護下
改善結構

> **這是實作時的短循環，不是 PR Review 之後才補測試。**

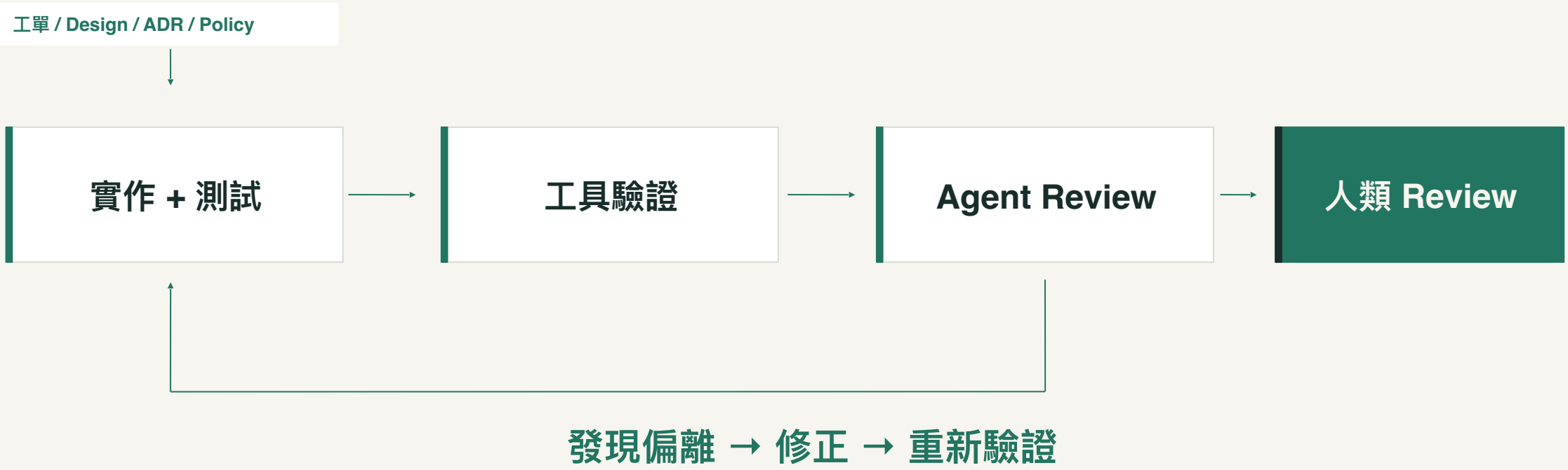
用測試守住行為，但不要流於形式

測試要有實際價值，而不是為了寫而寫。

續讀下一頁	[A, B] → [C, D]	驗收：沒有重複資料
抵達最後一頁	nextCursor 不存在	驗收：正確表示結束
共用層重構	既有 API 契約不變	回歸：保留原有行為

> **流於形式的測試全過 ≠ 行為正確。**

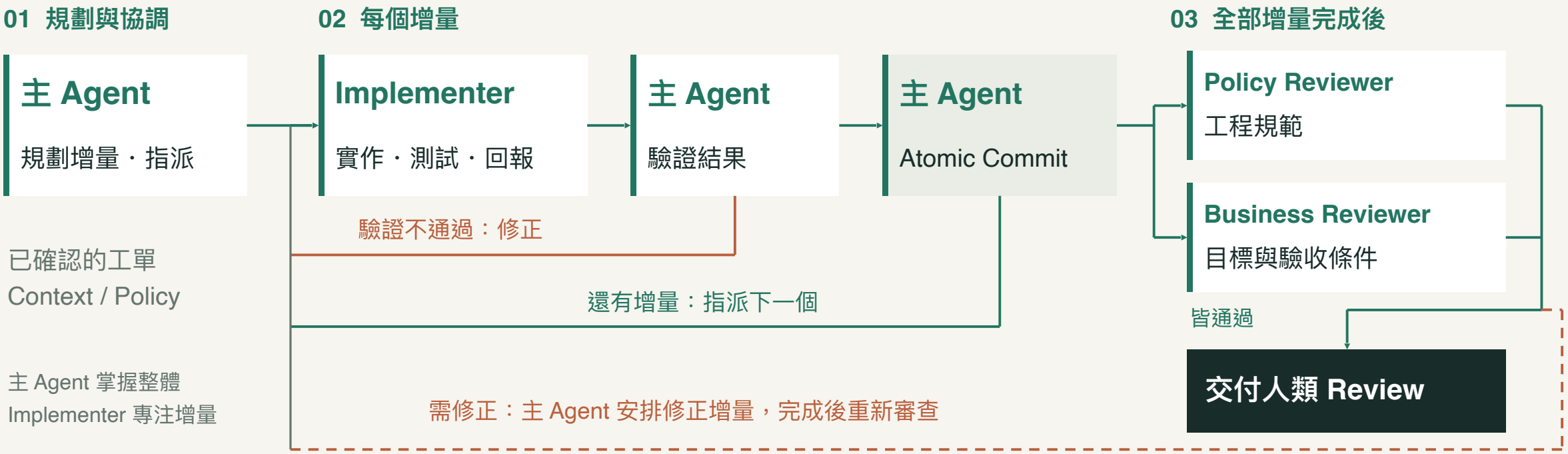
整合為回饋循環，讓 AI 可以持續修正



> 這些不是 AI 時代的新技術，而是 Agent 也能參與的工程紀律。

把實作拆成可驗證的增量，再逐步提交

我的實踐：atomic-implementation-loop



每個增量先驗證、再提交；整體審查後，仍由人類決定。

先診斷缺口，再決定要補什麼

三個角度，診斷 Agent 的工作環境。

Context

它知道
要做什麼嗎？

需求、語言、設計與決策

Constraints

它知道
不能怎麼做嗎？

適用的規則與責任邊界

Feedback

它知道
哪裡偏離了嗎？

測試、工具與 Review 訊號

先問缺了什麼，再對症下藥。

規則、方法、檢查與角色，需要不同載體

Policy	**規則**	Pagination 必須用共用層
Skill	**可重複的方法**	需求討論、TDD、Code Review
Tool	**確定性的檢查 / 執行**	Linter、Dependency Checker
Agent	**一段有邊界的職責**	讀取 PR、檢視、提出證據

> **不是四選一；同一個問題可以由多種機制共同保護。**

從小地方開始，不必一次設計完整系統

01 **需求 + 完成條件**

至少知道要做什麼、如何驗收

02 **基本 Policy**

最重要的工程規則與禁忌

03 **一致的驗證入口**

Lint / Type Check / Build / Test

04 **Git / CI 回饋**

讓檢查真的進入工作流程

> **從真實問題逐步補強，不必一次做到完整。**

當它能持續吸收教訓，
就成了一套工程系統。

這些能力，
為什麼只能服務我一個人？

從這裡開始，是團隊化的推導

團隊交付效率是要驗證的目標。

一人專案

真實經驗
與具體案例

重構、Policy、工具與回饋

團隊協作

共享、分發、治理
仍待導入驗證

共享可重用的能力，保留專案自己的規則

共通的部分集中維護，特定的 Context 與限制留在專案。

Policy	**共同規則**	+	Repository 規則
Skill	**可重用方法**	+	Domain-specific 方法
Tool	**檢查能力**	+	Repository 設定
Agent	**角色 / 模板**	+	專案 Context 與能力

範例：Dependency Checker 可以共用；各 Repo 保留自己的依賴規則。

讓下一個 Repo，從共享的能力開始

分發工具把能力送進工作環境，但不代替審查與治理。

Team Skill Library
共同維護的 Skills

Skills CLI

選擇 / 安裝 / 更新

Repository
只安裝需要的能力

其他資產

Repo template / 共用工具套件 / CI template

不需要每個 Repository 重複打造同一套能力。

共同資產加上專案延伸，標準才能持續演進

共同資產 + 專案延伸；不是中央強制規定、設計每一步。

共同資產

團隊審查、有人維護

專案延伸

根據 Domain 與風險調整

Local → **Validate** → **Review** → **Shared**

有共通價值，才提升為共享資產；採用後持續回饋。

目標是減少重複投入與重複踩坑。

從最近一次
「怎麼又犯同樣的錯」
開始。

先修正問題，再問：這個問題該用什麼方式將經驗保存下來？

Policy / Skill / Tool / Agent

AI 不會替我們
創造工程紀律。

塑造它，仍然是
軟體工程師的責任。

參考資料｜工作流、Context 與 Git

R1 **Superpowers — official repository**

需求討論、規劃、測試與 Review 的能力示例

https://github.com/obra/superpowers

R2 **Matt Pocock Skills — official repository**

需求對齊、to-spec、to-tickets 與相關工程技能

https://github.com/mattpocock/skills

R3 **Anthropic — Effective context engineering for AI agents**

長任務的 context 管理與 compaction 取捨

https://www.anthropic.com/engineering/effective-context-engineering-for-ai-agents

R4 **Git — githooks documentation**

pre-commit 觸發中止提交；本地 hook 可以被略過

https://git-scm.com/docs/githooks

參考資料｜邊界、分發、驗收與測試

R5 **Nx — Enforce Module Boundaries**

透過標籤與相依限制執行模組邊界檢查

https://nx.dev/docs/features/enforce-module-boundaries

R6 **Vercel Labs — Skills CLI**

選擇性安裝、列出及更新 Skills

https://github.com/vercel-labs/skills

R7 **Cucumber — Gherkin Reference**

Given / When / Then 表達前提、事件與預期結果

https://cucumber.io/docs/gherkin/reference/

R8 **Martin Fowler — Test Driven Development**

Red / Green / Refactor

https://martinfowler.com/bliki/TestDrivenDevelopment.html